



EVOLUTIONARY MODEL WITH ITERATIONS

AN EVOLUTIONARY AND ITERATIVE DEVELOPMENT PROCESS...

- Recognizes the reality of changing requirements
 - Capers Jones's research on 8000 projects: **40% of final requirements arrived after development had already begun**
- Promotes early risk mitigation:
 - Breaks down the system into mini-projects and focuses on the riskier issues first.
 - **“plan a little, design a little, and code a little”**
- Encourages all development participants to be involved earlier on, :
 - End users, Testers, integrators, and technical writers

EVOLUTIONARY MODEL WITH ITERATION

- “A complex system will be most successful if implemented in small steps... “retreat” to a previous successful step on failure... opportunity to receive some feedback from the real world before throwing in all resources... and you can correct possible errors...” [Tom Glib in Software Metrics](#)

EVOLUTIONARY MODEL WITH ITERATION

- Evolutionary iterative development implies that the requirements, plan, estimates, and solution evolve or are refined over the course of the iterations, rather than fully defined and “frozen” in a major up-front specification effort before the development iterations begin. Evolutionary methods are consistent with the pattern of unpredictable discovery and change in new product development.” **Craig Larman**

EVOLUTIONARY MODEL

- First develop the core modules of the software.
- The initial skeletal software is refined into increasing levels of capability: (Iterations)
 - By adding new functionalities in successive versions.

ACTIVITIES IN AN ITERATION

- Software developed over several “mini waterfalls”.
- The result of a single iteration:
 - Ends with delivery of some tangible code
 - An incremental improvement to the software --- leads to evolutionary development

EVOLUTIONARY MODEL WITH ITERATION

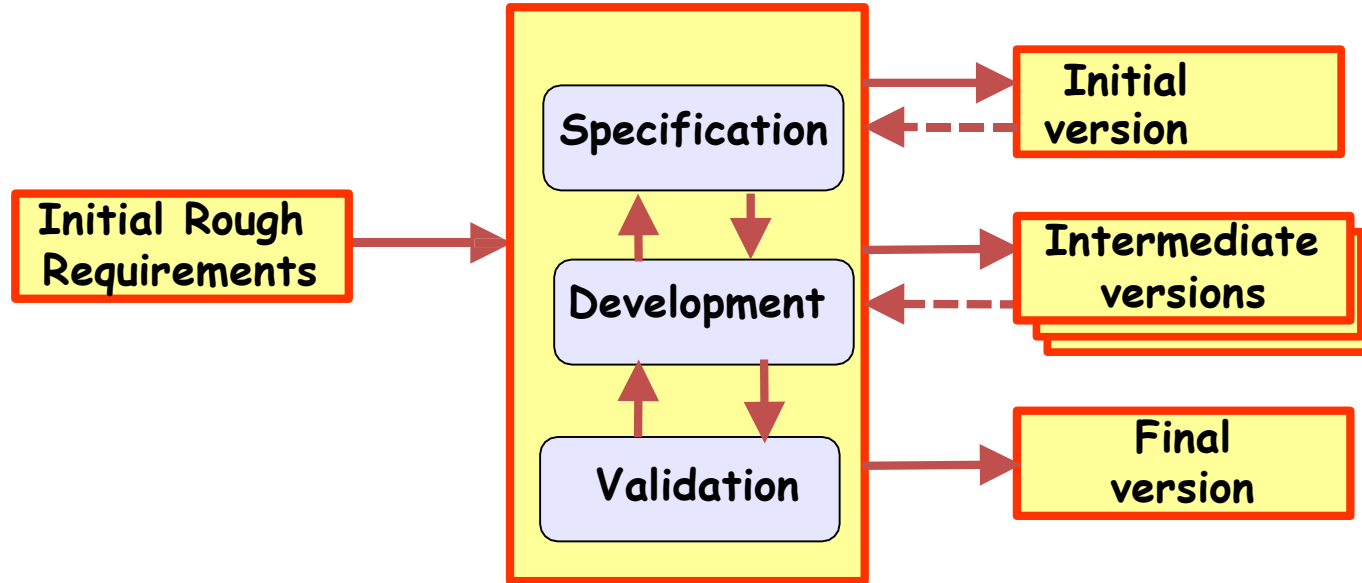
- Outcome of each iteration: tested, integrated, executable system
- Iteration length is short and fixed
 - Usually between 2 and 6 weeks
 - Development takes many iterations (for example: 10-15)
- Does not “freeze” requirements and then conservatively design :
 - Opportunity exists to modify requirements as well as the design...

EVOLUTIONARY MODEL (CONT.)

- Successive versions:
 - Functioning systems capable of performing some useful work.
 - A new release may include new functionality:
 - Also existing functionality in the current release might have been enhanced.

EVOLUTIONARY MODEL

- Evolves an initial implementation with user feedback:
 - **Multiple versions until the final version.**



ADVANTAGES OF EVOLUTIONARY MODEL

- Users get a chance to experiment with a partially developed system:
 - Much before the full working version is released,
- **Helps finding exact user requirements:**
 - Software more likely to meet exact user requirements.
- **Core modules get tested thoroughly:**
 - Reduces chances of errors in final delivered software.

ADVANTAGES OF EVOLUTIONARY MODEL

- Better management of complexity by developing one increment at a time.
- Better management of changing requirements.
- Can get customer feedback and incorporate them much more efficiently:
 - As compared when customer feedbacks come only after the development work is complete.

ADVANTAGES OF EVOLUTIONARY MODEL WITH ITERATION

- Training can start on an earlier release
 - customer feedback taken into account
- Frequent releases allow developers to fix unanticipated problems quicker.

EVOLUTIONARY MODEL: PROBLEMS

- **The process is intangible:**
 - No regular, well-defined deliverables.
- **The process is unpredictable:**
 - Hard to manage, e.g., scheduling, workforce allocation, etc.
- **Systems are rather poorly structured:**
 - Continual, unpredictable changes tend to degrade the software structure.
- **Systems may not even converge to a final version.**

RAD MODEL

RAPID APPLICATION DEVELOPMENT (RAD)

MODEL

- Sometimes referred to as the **rapid prototyping model**.
- Major aims:
 - Decrease the time taken and the cost incurred to develop software systems.
 - Facilitate accommodating change requests as early as possible:
 - Before large investments have been made in development and testing.

IMPORTANT UNDERLYING PRINCIPLE

- A way to reduce development time and cost, and yet have flexibility to incorporate changes:
 - **Make only short term plans and make heavy reuse of existing code.**

METHODOLOGY

- Plans are made for one increment at a time.
 - The time planned for each iteration is called a **time box**.
- Each iteration (increment):
 - Enhances the implemented functionality of the application a little.

METHODOLOGY

- During each iteration,
 - A quick-and-dirty prototype-style software for some selected functionality is developed.
 - The customer evaluates the prototype and gives his feedback.
 - The prototype is refined based on the customer feedback.

HOW DOES RAD FACILITATE FASTER DEVELOPMENT?

- RAD achieves fast creation of working prototypes.
 - Through use of specialized tools.
- These specialized tools usually support the following features:
 - **Visual style of development.**
 - **Use of reusable components.**
 - **Use of standard APIs (Application Program Interfaces).**

FOR WHICH APPLICATIONS IS RAD SUITABLE?

- Customized product developed for one or two customers only
- Performance and reliability are not critical.
- The system can be split into several independent modules.

FOR WHICH APPLICATIONS RAD IS UNSUITABLE?

- Few plug-in components are available
- High performance or reliability required
- No precedence for similar products exists
- The system cannot be modularized.

• Prototyping versus RAD

In prototyping model:

- The developed prototype is primarily used to gain insights into the solution
 - Choose between alternatives
 - Elicit customer feedback.
- The developed prototype:
- Usually thrown away.

PROTOTYPING VERSUS RAD

- In contrast:
 - In RAD the developed prototype evolves into deliverable software.
- RAD leads to faster development compared to traditional models:
 - However, the quality and reliability would possibly be poorer.

RAD VERSUS ITERATIVE WATERFALL MODEL

- In the iterative waterfall model,
 - All product functionalities are developed together.
- In the RAD model on the other hand,
 - Product functionalities are developed incrementally through heavy code and design reuse.
 - Customer feedback is obtained on the developed prototype after each iteration:
 - Based on this the prototype is refined.

RAD versus Iterative Waterfall Model

- The iterative waterfall model:
 - Does not facilitate accommodating requirement change requests.
- Iterative waterfall model does have some important advantages:
 - Use of the iterative waterfall model leads to production of good documentation.
 - Also, the developed software usually has better quality and reliability than that developed using RAD.

RAD VERSUS EVOLUTIONARY MODEL

- Incremental development:
 - Occurs in both evolutionary and RAD models.
- However, in RAD:
 - Each increment is a quick and dirty prototype,
 - Whereas in the evolutionary model each increment is systematically developed using the iterative waterfall model.
- Also, RAD develops software in shorter increments:
 - The incremental functionalities are fairly large in the evolutionary model.



UNIFIED PROCESS

7/18/2020

UNIFIED PROCESS

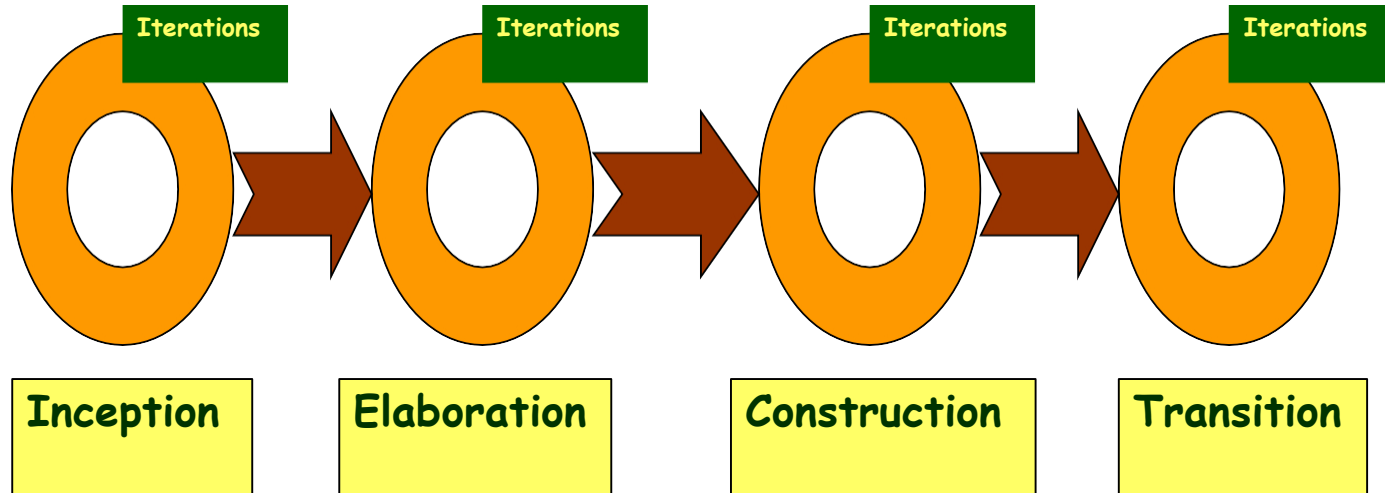
- Developed Ivar Jacobson, Grady Booch and James Rumbaugh
 - Incremental and iterative
- Rational Unified Process (RUP) is version tailored by Rational Software:
 - Acquired by IBM in February 2003.

FOUR PHASES --- AND ITERATIVE DEVELOPMENT AT EVERY PHASE

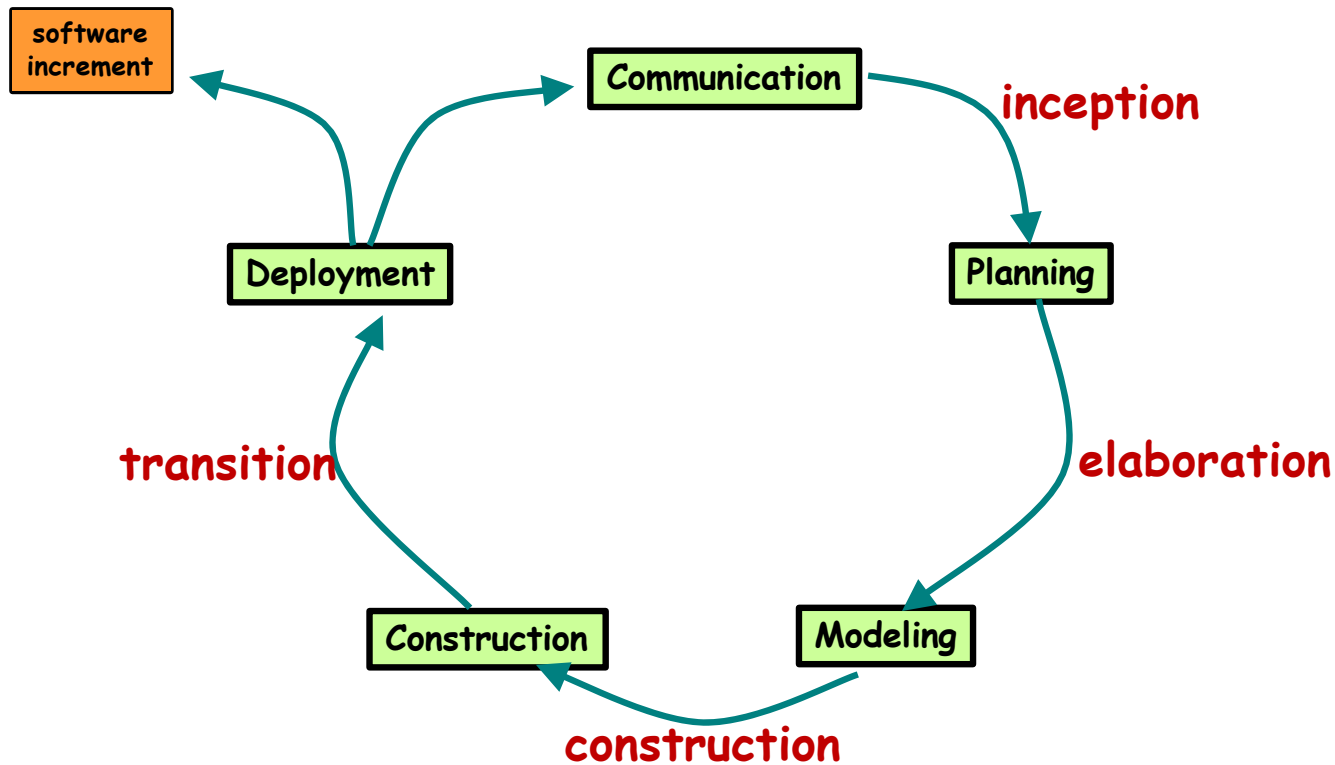
- Inception Phase
- Elaboration Phase
- Construction Phase
- Transition Phase

UNIFIED PROCESS ITERATIONS IN PHASES

The duration of and iteration may vary from two weeks or less.



UNIFIED PROCESS



UNIFIED PROCESS WORK PRODUCTS

Inception phase

vision document
initial use-case model
initial business case
initial risk list
project plan
prototype(s)
...

Elaboration phase

use-case model
requirements
analysis model
preliminary model
revised risk list
preliminary
manual
...

Construction phase

design model
SW components
test plan
test procedure
test cases
user manual
installation manual
...

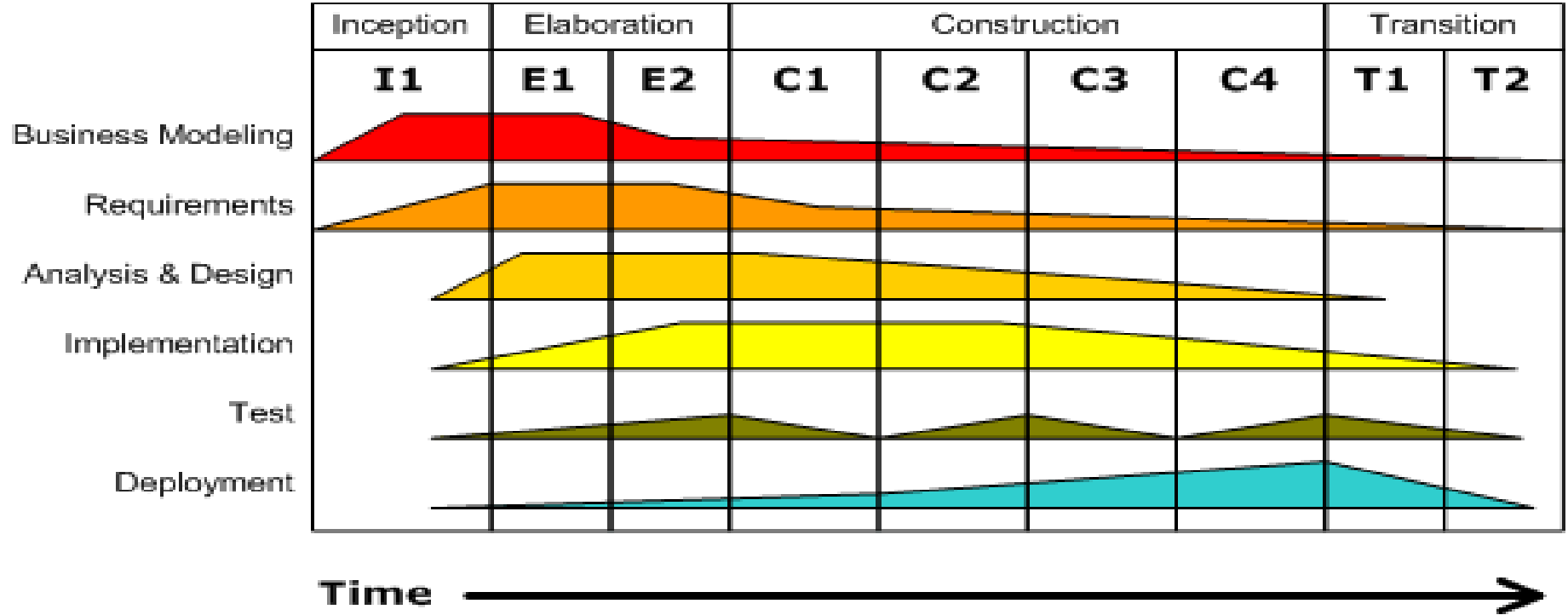
Transition phase

SW increment
beta test reports
user feedback
...

STRUCTURE OF RUP PROCESS

- Two dimensions.
- Horizontal axis:
 - Represents time and shows the lifecycle aspects of the process.
- Vertical axis:
 - Represents core process workflows.

TWO DIMENSIONS OF UNIFIED PROCESS



INCEPTION ACTIVITIES

- Formulate scope of project
- Risk management, staffing, project plan
- Synthesize a candidate architecture.

- Initial requirements capture
- Cost Benefit Analysis
- Initial Risk Analysis
- Project scope definition
- Defining a candidate architecture
- Development of a disposable prototype
- Initial Use Case Model (10% - 20% complete)
- First pass Domain Model

Outcome of Inception Phase

SPIRAL MODEL

- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
 - the innermost loop might be concerned with system feasibility,
 - the next loop with system requirements definition,
 - the next one with system design, and so on.
- There are no fixed phases in this model, the phases shown in the figure are just examples.

- The team must decide:
SPIRAL MODEL (CONT.)

- how to structure the project into phases.

- Start work using some generic model:

- add extra phases

- for specific projects or when problems are identified during a project.

- Each loop in the spiral is split into four sectors (quadrants).

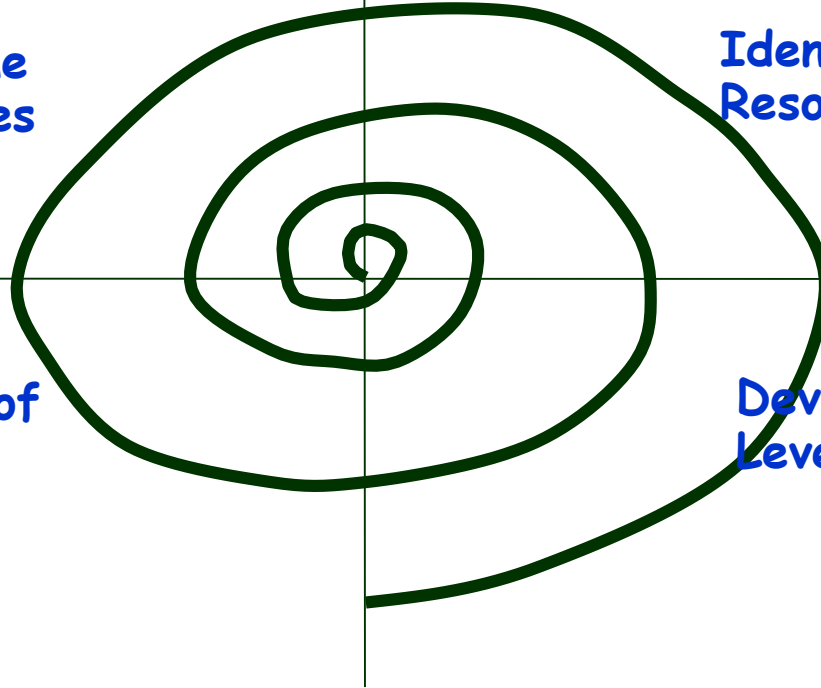
SPIRAL MODEL

Determine
Objectives

Identify &
Resolve Risks

Customer
Evaluation of
Prototype

Develop Next
Level of Product



Objective Setting (First Quadrant)

- Identify objectives of the phase,
- Examine the risks associated with these objectives.
 - Risk:
 - Any adverse circumstance that might hamper successful completion of a software project.
- Find alternate solutions possible.

RISK ASSESSMENT AND REDUCTION (SECOND QUADRANT)

- For each identified project risk,
 - a detailed analysis is carried out.
- Steps are taken to reduce the risk.
- For example, if there is a risk that requirements are inappropriate:
 - A prototype system may be developed.

- Development and Validation (**Third quadrant**):
 - develop and validate the next level of the product.
- Review and Planning (**Fourth quadrant**):
 - review the results achieved so far with the customer and plan the next iteration around the spiral.
- With each iteration around the spiral:
 - progressively more complete version of the software gets built.

- Subsumes all discussed models:
 - a single loop spiral represents waterfall model.
 - uses an evolutionary approach --
 - iterations over the spiral are evolutionary levels.
 - enables understanding and reacting to risks during each iteration along the spiral.
 - Uses:
 - prototyping as a risk reduction mechanism
 - retains the step-wise approach of the waterfall model.

**Spiral Model as
a Meta Model**



AGILE MODELS

7/18/2020

What is Agile Software Development?

- **Agile:** Easily moved, light, nimble, active software processes
- **How agility achieved?**
 - Fitting the process to the project
 - Avoidance of things that waste time

AGILE MODEL

- To overcome the shortcomings of the waterfall model of development.
 - Proposed in mid-1990s
- The agile model was primarily designed:
 - To help projects to adapt to change requests
- In the agile model:
 - The requirements are decomposed into many small incremental parts that can be developed over one to four weeks each.

IDEOLOGY: AGILE MANIFESTO

- Individuals and interactions *over*
 - process and tools
- Working Software *over*
 - comprehensive documentation
 - Responding to change *over*
 - following a plan

<http://www.agilemanifesto.org>

Agile Methodologies

- XP
- Scrum
- Unified process
- Crystal
- DSDM
- Lean

AGILE MODEL: PRINCIPAL TECHNIQUES

- **User stories:**
 - Simpler than use cases.
- **Metaphors:**
 - Based on user stories, developers propose a common vision of what is required.
- **Spike:**
 - Simple program to explore potential solutions.
- **Refactor:**
 - Restructure code without affecting behavior, improve efficiency, structure, etc.

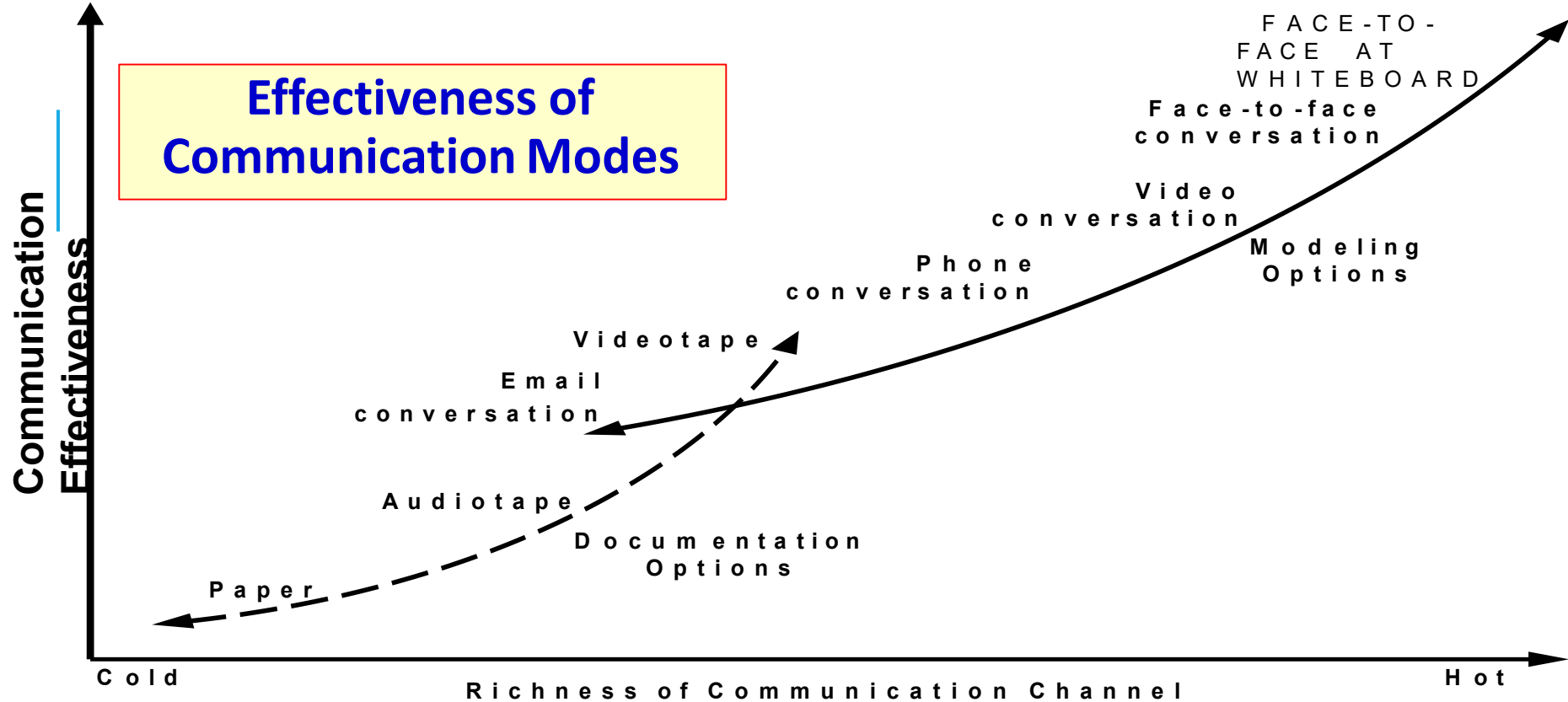
- At a time, only one increment is planned, developed, deployed at the customer site.

AGILE MODEL: NITTY GRITTY

- **No long-term plans are made.**
- An iteration may not add significant functionality,
 - But still a new release is invariably made at the end of each iteration
 - Delivered to the customer for regular use.

METHODOLOGY

- **Face-to-face communication favoured over written documents.**
- To facilitate face-to-face communication,
 - Development team to share a single office space.
 - Team size is deliberately kept small (5-9 people)
 - This makes the agile model most suited to the development of small projects.



Copyright 2002-2005 Scott W. Ambler
Original Diagram Copyright 2002 Alistair Cockburn

AGILE MODEL: PRINCIPLES

- The primary measure of progress:
 - **Incremental release of working software**
- Important principles behind agile model:
 - Frequent delivery of versions --- once every few weeks.
 - Requirements change requests are easily accommodated.
 - Close cooperation between customers and developers.
 - Face-to-face communication among team members.